

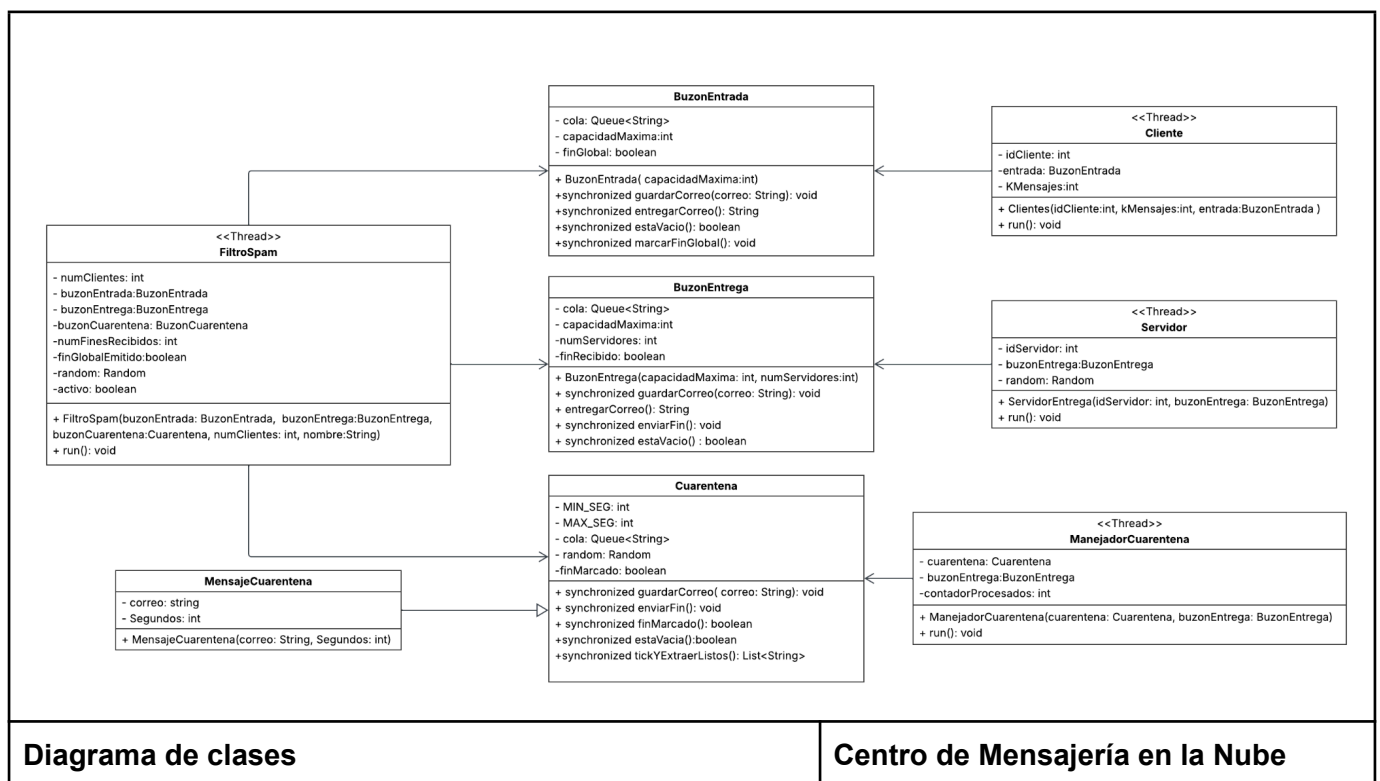
Universidad de los Andes

Caso 3-Concurrencia y Sincronización de Procesos

Sofía Rodríguez Chaparro 202222588

María Alejandra Sanabria Salazar 202326786

1. Diagrama de clases



Cliente

- La clase Cliente representa un hilo (Thread) que simula un usuario enviando correos electrónicos.
- Cada cliente tiene: Un identificador (idCliente), Un buzón de entrada (entrada, de tipo BuzónEntrada), Y una cantidad de mensajes a enviar (kMensajes).
- El método run() genera mensajes y los envía al buzón de entrada. Esto representa la generación continua de correos electrónicos que entran al sistema.

BuzónEntrada

- BuzónEntrada actúa como una cola sincronizada donde los clientes depositan los correos que generan.
- Sus atributos incluyen: Una cola (Queue<String>), Una capacidad máxima, y un indicador de fin global (finGlobal).

- Sus métodos controlan la concurrencia: `guardarCorreo()` agrega correos, `entregarCorreo()` los extrae, `estaVacio()` verifica si hay mensajes, `marcarFinGlobal()` señala que todos los clientes han terminado.

FiltroSpam

- Un `FiltroSpam` emite el FIN global únicamente cuando se cumplen simultáneamente estas condiciones: el buzón de entrada está vacío, la cuarentena está vacía (no quedan mensajes con TTL pendiente) y ya no se producirán nuevos mensajes porque se recibieron todos los FIN de clientes. En ese instante, se invoca a `buzonEntrega.enviarFin()`, que replica un FIN por cada servidor, y a `buzonCuarentena.enviarFin()` para que el manejador de cuarentena termine cuando su estructura quede vacía. La emisión del FIN global se garantiza una sola vez mediante una bandera compartida; los filtros terminan su ejecución al detectar que el FIN global ya fue emitido.

Cuarentena

- La clase `Cuarentena` representa un almacenamiento temporal para mensajes sospechosos. Guarda mensajes en espera durante un tiempo aleatorio (entre `MIN_SEG` y `MAX_SEG`).
- Sus métodos sincronizados permiten: Guardar correos (`guardarCorreo()`), Enviar señal de fin (`enviarFin()`), Extraer mensajes listos para ser reenviados (`tickYExtraerListos()`).

ManejadorCuarentena

- `ManejadorCuarentena` es otro hilo que se encarga de revisar periódicamente los mensajes en cuarentena. Toma los mensajes que ya cumplieron su tiempo y los envía al `BuzonEntrega`, simulando que han sido liberados tras revisión.
- Este hilo ejecuta su lógica dentro de `run()`.

BuzonEntrega

- `BuzonEntrega` es la cola donde los correos listos (no spam o revisados) se depositan antes de ser entregados.
- Mantiene: Una cola sincronizada, Capacidad máxima, Número de servidores que entregan correos, y un indicador de fin.
- Provee métodos como `guardarCorreo()`, `entregarCorreo()`, `enviarFin()` y `estaVacio()`.

Servidor

- La clase `Servidor` también es un hilo que simula los servidores encargados de entregar los correos finales.
- Cada servidor: Tiene un identificador (`idServidor`), Accede al `BuzonEntrega`, Usa un objeto `Random` para simular tiempos aleatorios de entrega.
- El método `run()` se encarga de procesar los correos hasta que recibe la señal de fin.
- Cada `Servidor de Entrega` inicia su operación al encontrar un mensaje de inicio (START). Antes de ese evento, el servidor permanece en espera activa sin procesar datos. Cada servidor finaliza al leer su propio FIN.

MensajeCuarentena

- Finalmente, MensajeCuarentena representa un correo que ha sido puesto en cuarentena junto con el tiempo que debe permanecer allí antes de liberarse.
- Tiene los atributos: correo (el texto del mensaje), segundos (el tiempo de espera asignado).

2. Funcionamiento del programa

El centro de mensajería opera mediante una arquitectura basada en hilos productores y consumidores que se comunican a través de distintos buzones.

En primer lugar, los clientes emisores funcionan como hilos productores. Cada uno se inicia con un número fijo de correos electrónicos que debe generar una cantidad definida en el archivo de configuración. Al comenzar su ejecución, cada cliente envía un mensaje de inicio al sistema y luego produce sus correos, asignando a cada uno un identificador único y un indicador de spam generado aleatoriamente. El identificador se forma combinando el número del cliente con un contador secuencial, una vez completada la generación de todos los correos asignados, el cliente emisor envía un mensaje de fin para indicar que ha terminado su producción.

El buzón de entrada actúa como un espacio compartido de capacidad limitada donde los clientes depositan sus correos, si el buzón está lleno, los emisores deben esperar hasta que haya espacio disponible, implementando así una espera pasiva y cuando un cliente logra depositar un mensaje, despierta a uno de los filtros de spam para que inicie su análisis.

Los filtros de spam operan como hilos analistas que consumen mensajes del buzón de entrada también en espera pasiva, cada filtro evalúa si un correo es spam o si es válido. Si un mensaje es identificado como spam, se envía al buzón de cuarentena (con capacidad ilimitada) bajo espera semiactiva, asignándole un tiempo aleatorio entre 10000 y 20000 unidades que representa su permanencia en cuarentena, pero si el mensaje es válido, el filtro lo envía al buzón de entrega. Los mensajes de inicio y fin no se consideran spam, ya que los primeros permiten a los filtros conocer el número de clientes activos, y los segundos les indican cuándo ha concluido la generación de mensajes. Una vez que todos los correos han sido analizados y enviados al buzón de entrega, uno de los filtros coloca allí un mensaje de fin, pero solo cuando el buzón de entrada y el de cuarentena están vacíos y no se producirán nuevos mensajes. Los filtros terminan su ejecución al recibir tantos mensajes de fin como clientes haya en el sistema y después de haber enviado los mensajes de finalización correspondientes al buzón de entrega y al de cuarentena.

El manejador de cuarentena se encarga de consumir, en espera semiactiva, los mensajes depositados en el buzón de cuarentena, este componente revisa periódicamente todos los mensajes, reduciendo en una unidad el contador de tiempo asociado a cada uno. Cuando el contador llega a cero, el mensaje se transfiere al buzón de entrega. Adicionalmente, el manejador simula la eliminación de mensajes maliciosos, mediante cada revisión, donde se genera un número aleatorio entre 1 y 21, y si dicho número es múltiplo de 7, el

mensaje se descarta. El manejador ejecuta este proceso cada segundo y finaliza su ejecución al recibir un mensaje de fin.

El buzón de entrega recibe todos los mensajes válidos provenientes de los filtros de spam y del manejador de cuarentena, su función principal es distribuir los correos hacia los servidores de entrega, una vez que no quedan mensajes pendientes ni se producirán nuevos, el buzón genera y envía un mensaje de fin a los servidores. Tras emitir el mensaje, el buzón de entrega replica este mensaje de fin para que todos los servidores puedan cerrar sus procesos correctamente.

Finalmente, los servidores de entrega son hilos consumidores que procesan los mensajes válidos tomados del buzón de entrega. Cada servidor lee los correos uno a uno en espera activa, simulando el procesamiento mediante una pausa de duración aleatoria antes de pasar al siguiente mensaje. Estos servidores continúan operando hasta recibir un mensaje de fin, momento en el cual concluyen su ejecución.

En conjunto, este sistema coordina la generación, clasificación, cuarentena y entrega de correos electrónicos simulando un entorno concurrente, donde cada componente cumple un rol específico que garantiza el flujo ordenado de los mensajes hasta su procesamiento final.

3. Validación / Pruebas

Escenario:

```
int numMensajes = 100;
```

```
int numClientes = 10;
```

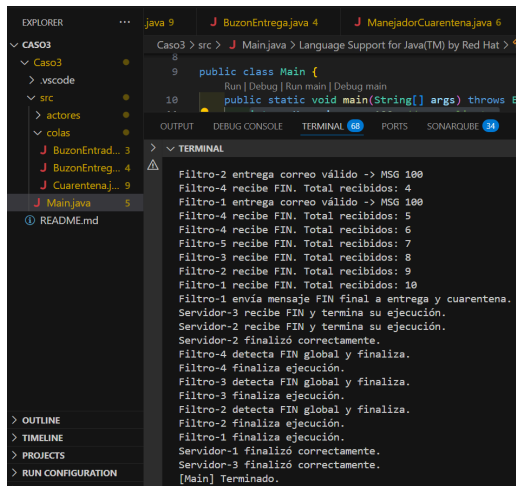
```
int numFiltros = 5;
```

```
int numManejadores= 1;
```

```
int numServidores = 3;
```

```
int capEntrada = 20;
```

```
int capEntrega = 20;
```



```
public class Main {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) throws Ex...
```

```
Filtro-2 entrega correo válido -> MSG 100
Filtro-4 recibe FIN. Total recibidos: 4
Filtro-1 entrega correo válido -> MSG 100
Filtro-4 recibe FIN. Total recibidos: 5
Filtro-4 recibe FIN. Total recibidos: 6
Filtro-5 recibe FIN. Total recibidos: 7
Filtro-3 recibe FIN. Total recibidos: 8
Filtro-2 recibe FIN. Total recibidos: 9
Filtro-1 recibe FIN. Total recibidos: 10
Filtro-1 envía mensaje FIN final a entrega y cuarentena.
Servidor-3 recibe FIN y termina su ejecución.
Servidor-2 recibe FIN y termina su ejecución.
Servidor-2 finalizó correctamente.
Filtro-4 detecta FIN global y finaliza.
Filtro-4 finaliza ejecución.
Filtro-3 detecta FIN global y finaliza.
Filtro-3 finaliza ejecución.
Filtro-2 detecta FIN global y finaliza.
Filtro-2 finaliza ejecución.
Filtro-1 finaliza ejecución.
Servidor-1 finalizó correctamente.
Servidor-3 finalizó correctamente.
[Main] Terminado.
```

Resultado:

la entrada nunca supera su capacidad (productores bloquean en espera pasiva), los filtros clasifican correctamente, los mensajes spam reciben TimeToLive entre 10 y 20, el manejador reduce TimeToLive cada 1 s y descarta 1 de cada 7 dentro de los primeros 21 procesados, el resto llega a entrega; los servidores leen en espera activa y finalizan tras su FIN. Al terminar, entrada, entrega y cuarentena quedan vacías y todos los hilos finalizan.

Escenario:

int numMensajes = 2;

int numClientes = 10;

int numFiltros = 5;

int numManejadores= 1;

int numServidores = 3;

int capEntrada = 20;

int capEntrega = 20;

Resultado:

```
c:\Users\User\Downloads\Caso3> c: && cd c:\Users\User\Downloads\Caso3 && cmd /C "C:\Users\User\AppData\Roaming\Code\User\globalStorage\pleiades.java-extension-pack-jdk\java\latest\bin\java.exe --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\User\AppData\Roaming\Code\User\workspaceStorage\c485756dbb3a5df874e6d7943a69ab04\redhat.java\jdt_ws\Caso3_338d72ce\bin Main "
```

```
Cliente-2 envía: MSG 1
Filtro-1 detecta inicio de cliente -> INICIO 7
Cliente-7 envía: MSG 1
Cliente-1 envía: MSG 1
Filtro-3 detecta inicio de cliente -> INICIO 2
Filtro-4 detecta inicio de cliente -> INICIO 6
Cliente-0 envía: MSG 1
Cliente-6 envía: MSG 1
Cliente-4 envía: MSG 1
Filtro-5 entrega correo válido -> MSG 1
Cliente-5 envía: MSG 1
Cliente-8 envía: MSG 1
Cliente-2 envía: MSG 2
Filtro-2 detecta SPAM -> MSG 1
Filtro-1 detecta SPAM -> MSG 1
Cliente-3 envía: MSG 1
Cliente-9 envía: MSG 1
Cliente-7 envía: MSG 2
Cliente-1 envía: MSG 2
```

```
Filtro-3 recibe FIN. Total recibidos: 5
Filtro-3 recibe FIN. Total recibidos: 6
Filtro-2 recibe FIN. Total recibidos: 7
Filtro-4 recibe FIN. Total recibidos: 8
Filtro-5 recibe FIN. Total recibidos: 9
Filtro-1 recibe FIN. Total recibidos: 10
Filtro-1 envía mensaje FIN final a entrega y cuarentena.
Filtro-1 finaliza ejecución.
Filtro-3 detecta FIN global y finaliza.
Filtro-2 detecta FIN global y finaliza.
Filtro-2 finaliza ejecución.
Servidor-3 recibe FIN y termina su ejecución.
Filtro-4 detecta FIN global y finaliza.
Filtro-4 finaliza ejecución.
Filtro-5 detecta FIN global y finaliza.
Servidor-1 recibe FIN y termina su ejecución.
Servidor-1 finalizó correctamente.
Servidor-2 recibe FIN y termina su ejecución.
Filtro-3 finaliza ejecución.
Servidor-3 finalizó correctamente.
Filtro-5 finaliza ejecución.
Servidor-2 finalizó correctamente.
[Main] Terminado.
```

Resultados: se observa backpressure controlado en entrada y entrega. La gestión de TimeToLive es estable, el FIN global se emite una sola vez cuando entrada y cuarentena quedan vacías, se replican FIN suficientes (uno por servidor) y la terminación es ordenada.

Criterio de aceptación: en ambos casos no se violan capacidades, no hay interbloqueos y la condición de finalización se cumple tal como está definida.

5. Conclusión

En conclusión, el sistema del centro de mensajería implementa una arquitectura concurrente eficiente que coordina de forma ordenada la producción, clasificación, cuarentena y entrega de correos electrónicos mediante el uso de hilos y buzones con distintos modos de espera. Cada componente, desde los clientes emisores hasta los servidores de entrega, cumple un rol esencial que asegura la sincronización y el flujo continuo de los mensajes, evitando pérdidas o bloqueos. Gracias al uso de mecanismos de espera pasiva y semiactiva, así como al manejo controlado de los mensajes de inicio y fin, el sistema logra simular de manera realista un entorno de mensajería con detección de spam y procesamiento distribuido, garantizando una terminación ordenada y el correcto funcionamiento de todos los procesos involucrados.